11_claude_toolkit

claude_toolkit ⇄ HelixAgent provider auto-detection — research & design

Date: 2026-07-06 **Scope:** design how the SIBLING project `claude_toolkit` (`/home/milos/Factory/projects/tools_and_research/claude_toolkit`, OUTSIDE the helix_code monorepo) auto-recognises **HelixAgent** as an LLM provider when the `helixagent` binary is on the system `PATH`, registers a HelixAgent provider alias exposing **all** HelixAgent-served models (the AI Debate Ensemble, HelixLLM, and any other `/v1/models` entries), validates that alias with a comprehensive real-request test like every other provider alias, and updates + re-incorporates its vendored **LLMsVerifier**. **Method:** read the actual claude_toolkit + helix_agent source (no guessing, §11.4.6). All file/line citations below are FACTs from the current checkout.

---

1. Current-state map — how claude_toolkit registers providers + aliases TODAY

1.1 The resolution pipeline (env-key-driven, NOT PATH-driven)

The provider system is entirely **API-key-variable-NAME driven**. The grounding fact from the inventory is confirmed: a provider only comes into existence because an **env-key variable name** is present in the keys file — there is no binary/PATH detection anywhere in the pipeline.

Entry point: `scripts/claude-providers.sh` (the `sync` subcommand is default).

Pipeline (`cmd_sync`, lines 170–266):

1. **`present_key_vars()`** (`claude-providers.sh:129-156`) — extracts API-key variable **NAMES** (never values) from the keys file (`$CMA_KEYS_FILE`, default `~/api_keys.sh`) by grepping `^[[:space:]]*(export )?VAR=` lines, then sources the file in a subshell and keeps only names whose VALUE is non-empty. Output: a list of key-var names.
2. **`ensure_catalog()`** (`:88-126`) — fetches + caches the **models.dev** catalog (`https://models.dev/api.json`) to `~/.local/share/claude-multi-account/providers/models.dev.cache.json` (24 h TTL; offline uses cache).

3. **`resolve_records()`** (:161–167) → runs **`providers_resolve.py`** with `--models-dev CACHE --keys "A_API_KEY,B_API_KEY,…" --key-aliases … --overrides ….`
4. For each `resolved` record: optionally verify (§1.4), then write a per-provider **env file** + **shell alias** + **config dir** (§1.3).

1.2 `providers_resolve.py` — "the dynamic brain" (pure, offline)

`scripts/providers_resolve.py` (244 lines) maps each key-var name → a concrete provider record. Key mechanics:

- **`classify(key_var)`** (:54–64) — a key-var name is `llm`, `vcs`, or `infra` by regex on the NAME (e.g. `^GITHUB_` → vcs, `^FIREBASE` → infra, `GITHUB_MODELS` exempt → llm). Only `llm` keys become aliases.
- **`find_provider_by_env(catalog, key_var)`** (:67–73) — matches the key-var name against each models.dev provider's `env` array; OR **`key_aliases.get(key_var)`** short-circuits it (:165).
- **`select_models(provider)`** (:89–137) — picks `(strong, fast, context_limit, max_output)` from the models.dev metadata (reasoning → newest → largest context → highest output-cost for strong; lowest input-cost tool-capable for fast).
- **`transport_for(provider)`** (:76–83) — `native` iff the provider's models.dev `npm`/`api` ends with `anthropic`, else `router`.
- **`overrides.json`** per-provider pins override `alias / base_url / transport / strong_model / fast_model` (:190–193).
- A `router` record **with no `base_url`** is downgraded to `unmapped` (:198–203) — it cannot be configured for `ccr`.

Record shape (:22–27): `{key_var, classification, provider_id, alias, base_url, transport, strong_model, fast_model, context_limit, max_output, status, reason}`; `status ∈ {resolved, unmapped, skipped}`.

1.3 The two human-editable config files — where aliases "live"

`scripts/providers/`:

- **`key-aliases.json`** — maps a key-var NAME → a models.dev provider id (name normalisation). Current content: `ZAI_API_KEY→zai-coding-plan`, `CODESTRAL_API_KEY→mistral`, `POE_API_KEY→poe`, etc.
- **`overrides.json`** — per-provider manual pins (transport/base_url/models). E.g. `deepseek` pinned to `transport:native, base_url: https://api.deepseek.com/anthropic`; `kimi-for-coding` pinned to a native `/coding/` endpoint. **This is the file where a local, non-models.dev provider gets its base_url + models pinned.**

Adding a provider mapping without editing code: `claude-providers add --from-key VAR --id PROVIDER_ID` (`cmd_add`, `:385–402`) writes the `key-aliases.json` entry then re-runs `sync`.

1.4 What a "provider alias" physically IS

Per resolved provider, three artefacts are written by `lib.sh` helpers:

- **env file** `~/.local/share/claude-multi-account/providers/<id>.env` (`cma_provider_write_env`, `lib.sh:898`) — **non-secret** fields only: `CMA_PROVIDER_ID / KEYVAR / TRANSPORT / BASE_URL / MODEL / FAST_MODEL / CONFIG_DIR / CONTEXT_LIMIT / MAX_OUTPUT / ALIAS`. The secret is NEVER written; only the key-var **NAME** is stored, resolved at launch.
- **shell alias** in `~/.local/share/claude-multi-account/aliases.sh` (`cma_provider_write_alias`, `lib.sh:954`): `alias <name>="cma_run_provider <id>"`.
- **config dir** `~/.claude-prov-<id>` with all shared plugins symlinked (`cma_link_shared_items`, `lib.sh:801`).

`cma_run_provider()` (`lib.sh:476`) is the launch wrapper: it sources the env file, reads the secret from the named key-var at run time, sets `ANTHROPIC_BASE_URL / ANTHROPIC_AUTH_TOKEN / ANTHROPIC_MODEL` (native transport) or routes through `ccr` (router transport), and launches Claude Code — gated on verified/activation status.

1.5 Verification (the multi-layer adapter)

`providers-verify.sh` (`scripts/providers-verify.sh`) is the pluggable adapter, tried in order:

1. **LLMsVerifier binary** — if `submodules/LLMsVerifier/bin/model-verification` is executable, run `--provider … --model … --verbose`; PASS iff stdout has `Status: verified` AND `Can See Code: true` (`:46–52`).
2. **HTTP probe** — else `curl {BASE_URL}/models` with a bearer token passed via `--config <(printf …)` (never argv); `200` → verified, `401/403` → failed, else → unverified (`:56–68`).
3. Else `unverified` (not a failure; alias still usable).

Layer 3 = **semantic code-visibility** (`providers-semantic.sh`), layer 4 = **live superpowers-TUI** (`verify_superpowers_tui.sh`). `cmd_sync` records the verdict + first failing layer into the status cache (`claude-providers.sh:226–260`).

1.6 How existing provider aliases are TESTED (the mirror to copy)

- **`scripts/tests/verify_providers_live.sh`** (181 lines, read-only, SKIP when no aliases installed) — the canonical live proof: asserts every `<id>.env` has required non-secret fields, **contains NO secret values** (structural: only `CMA_PROVIDER_*=` lines), each alias resolves to `cma_run_provider`, and runs layer-3 semantic + layer-4 superpowers per installed provider, writing raw evidence to `scripts/tests/proof/` with a secret-redactor. Includes a **negative case** (a neutral prompt must NOT match the engagement marker) — the anti-bluff seam (§11.4.107(10)).
- **`scripts/tests/test_providers.sh`** (71 KB) — the hermetic unit/integration suite (uses a fake `models.dev` catalog + a `fakebin` PATH + a fake keys file).
- **`scripts/alias_e2e_test.py`**, **`scripts/tests/verify_aliases_live.sh`** — end-to-end alias exercises.
- Evidence lives in `scripts/tests/proof/*.txt` (e.g. `50-providers-live.txt`, `60-xiaomi-live.txt`, `70-zen-live.txt`).

---

2. The vendored LLMsVerifier — path, version, incorporation, update

2.1 Where + how incorporated

- **Git submodule** declared in `.gitmodules`: `path = submodules/LLMsVerifier`, `url = git@github.com:vasic-digital/LLMsVerifier.git`, `branch = main`.
- **Go module** lives one level down at `submodules/LLMsVerifier/llm-verifier/` — `go.mod` module id **`digital.vasic.llmsverifier`** (the same module helix_agent imports as `digital.vasic.llmsverifier/pkg/cliagents`).
- Build targets under `llm-verifier/cmd/`: `code-verification`, `model-verification`, `full-verify`, `quick-verify`, `crush-config-converter`, `partners`, …
- **Consumers inside claude_toolkit:**
  - `providers-verify.sh` expects a prebuilt binary at `submodules/LLMsVerifier/bin/model-verification` (strategy 1).
  - `claude-verify-providers.sh` **builds on demand** (cached): `go build -o .local-cache/code-verification ./cmd/code-verification/` inside `llm-verifier/`, rebuilding when `cmd/code-verification/main.go` is newer than the cached binary (`:57–63`); runs with `--config code_verification_config.json`.

- `claude-semantic-visibility.sh` / `providers-semantic.sh` build + run the semantic-code-visibility command (layer 3).
- `config/containers/llmsverifier/` — a container build (`Dockerfile.nezha` copies `LLMsVerifier/llm-verifier/`) + a Grafana dashboard.

2.2 Current version / drift (FACT, captured 2026-07-06)

- Vendored HEAD: `17b4bfb6` (`merge: reconcile v1.12.1 (I-1 exit-3 + CONST-069) with origin/main`), `git describe` → `helixcode-v1.1.0-13-g17b4bfb6`.
- `git fetch origin` → **origin/main** = **`0e7d6949`**, local is **1 commit behind** (`git rev-list --count HEAD..origin/main` = 1).

2.3 Update + re-incorporate procedure

```
cd claude_toolkit
git fetch --all --prune                                  # §11.4.60 fetch-before-edit
cd submodules/LLMsVerifier
git fetch origin --prune --tags
git log --oneline HEAD..origin/main                      # investigate the delta (§11.4.71)
git checkout main && git merge --ff-only origin/main # merge-onto-latest-main (§11.4.113); NO force
cd ../..
# Re-incorporate: bump the parent gitlink + rebuild the consumed binaries
git add submodules/LLMsVerifier
# Rebuild caches so the new codebase is actually exercised (§11.4.108 artifact layer):
rm -f .local-cache/code-verification                     # force rebuild on next run
scripts/claude-verify-providers.sh --help >/dev/null # triggers cached go build
( cd submodules/LLMsVerifier/llm-verifier \
  && go build -o ../bin/model-verification ./cmd/model-verification/ )   # strategy-1 binary
# Validate: run the toolkit's own verify suites against the rebuilt binary
bash scripts/tests/test_verify_providers.sh
bash scripts/tests/verify_providers_live.sh              # captures proof/*.txt
git commit -m "chore(LLMsVerifier): bump submodule to <sha> + rebuild verifier binaries"
```

Notes: (a) `test_verify_providers.sh:23` asserts `.gitmodules` declares `submodules/LLMsVerifier`, so the submodule wiring is gate-protected; (b) the Go toolchain is required (`claude-verify-providers.sh:47`); (c) a `bin/model-verification` prebuilt binary is what strategy-1 wants but it is NOT currently committed — the re-incorporation SHOULD build it (or the toolkit falls back to strategy-2 HTTP probe / on-demand build).

---

3. HelixAgent current model surface (FACT, from helix_agent source)

Source: `helix_code/submodules/helix_agent/cmd/helixagent/main.go`.

- **Binary name: `helixagent`** (built from `cmd/helixagent/`).
- Serves an **OpenAI-compatible HelixLLM REST API** at `http://<HELIXAGENT_HOST|localhost>:<PORT|8100>/v1` — endpoints **`/v1/models`**, **`/v1/chat/completions`** (`main.go:2564`, `:3137` comments; base = `fmt.Sprintf("http://%s:%s", host, port)` at `:2404–2413`, default host `localhost`, default `PORT=8100`, `ServerPort:"8100"` at `:1321`).
- A **native HelixLLM endpoint** at `https://localhost:8443` (self-signed TLS, auto-trust configured by `configureHelixLLMTLS`, `:2027`), env `HELIX_LLM_ENDPOINT` / `HELIX_LLM_API_KEY`.
- **Auth:** `HELIXAGENT_API_KEY` (bearer / API key; `--generate-api-key` generates one, `--api-key-env-file` writes it; `:2385`, `:87`, `:94`).
- **Exposed models** (from `--generate-opencode-config`, `:2437–2447+`): a single provider **"Helix Agent"** (`helixagent`, npm `@ai-sdk/openai-compatible`, `BaseURL: baseURL+"/v1"`, `APIKey: apiKey`) with models **`helix-debate`** ("Helix AI Debate Ensemble", ctx 128000/out 8192) and **`helix-llm`** ("fast local inference via HelixLLM"). The authoritative, non-hardcoded enumeration is the live **`GET /v1/models`** response.
- Relevant CLI flags: `--generate-api-key`, `--generate-opencode-config`, `--generate-crush-config`, `--generate-agent-config <agent>`, `--list-agents`, `--version`.

---

4. DESIGN

Because the whole pipeline is keyed on an **env-key variable name**, HelixAgent (a local binary with no cloud API key) needs a **new PATH-detection path** that (a) synthesises a provider record without a models.dev entry, (b) enumerates models from HelixAgent's live `/v1/models`, and (c) supplies base_url + a key-var so the existing env/alias/verify machinery works unchanged.

4.a PATH-detection mechanism → register a HelixAgent provider + alias

Add a small, self-contained detector invoked once at the top of `cmd_sync` (and `cmd_sync_multi`), BEFORE `resolve_records`. Keep it decoupled (§11.4.28): no HelixAgent-specific logic inside `providers_resolve.py` (that stays pure / models.dev-only) — the detector produces a `resolved`-shaped record and feeds it through the SAME `cma_provider_write_env` / `cma_provider_write_alias` / verification code path.

New helper `detect_helixagent()` (in `lib.sh` or a new `scripts/providers/detect_local.sh`), pseudo-contract:

```
# Is HelixAgent on PATH? (config-driven binary name, default
        "helixagent")
: "${CMA_HELIXAGENT_BIN:=helixagent}"
: "${CMA_HELIXAGENT_ID:=helixagent}"
: "${CMA_HELIXAGENT_HOST:=localhost}"
: "${CMA_HELIXAGENT_PORT:=8100}"
: "${CMA_HELIXAGENT_KEYVAR:=HELIXAGENT_API_KEY}"   # the key-var
        NAME (not value)

detect_helixagent() {
  command -v "$CMA_HELIXAGENT_BIN" >/dev/null 2>&1 || return 1
        # PATH gate
  local base="http://${CMA_HELIXAGENT_HOST}:$
        {CMA_HELIXAGENT_PORT}/v1"
  # Health/liveness: is the server actually up? (honest — a
        binary on PATH
  # is not a running server, §11.4.6). If down, register alias
        but mark
  # 'unverified' rather than fabricate a model list.
  ...
}
```

Two design sub-decisions:

1. **Detection is by PATH, but the record still flows as a `resolved` JSON record** so the dedupe-by-`provider_id`, env-write, alias-write, and verification loop in `cmd_sync` (:197–262) is reused verbatim. The detector emits ONE extra TSV/JSON record appended to `resolve_records` output. Concretely: `resolve_records()` becomes `resolve_records() { python3 "$RESOLVER" …; detect_helixagent_record; }` merged via `jq -s add`.
2. **transport = `native` vs `router`.** HelixAgent's `/v1` is **OpenAI-compatible**, NOT Anthropic-native, so `cma_run_provider`'s native path (which sets `ANTHROPIC_BASE_URL`) would NOT work directly for Claude Code. Therefore

the HelixAgent alias MUST use **`transport: router`** so the toolkit routes through `ccr` (claude-code-router), which translates Anthropic↔OpenAI — exactly as every other OpenAI-style provider alias does. (If HelixAgent later exposes an Anthropic-native `/anthropic` endpoint like deepseek's override, a future `overrides.json` pin can promote it to `native`.)

To keep it override-driven (no hardcoding, §11.4.6 / CONST-045-style), ship a default `overrides.json` block plus the PATH gate:

```
"helixagent": {
  "transport": "router",
  "base_url": "http://localhost:8100/v1",
  "strong_model": "helix-debate",
  "fast_model": "helix-llm"
}
```

…but the base_url/models are **discovered** at detection time (4.b) and only fall back to these pins when discovery fails — so a running HelixAgent with a different port/model set is honoured, and a down one still yields a usable (unverified) alias.

4.b Enumerating HelixAgent's exposed models (all of them)

Query the live OpenAI-compatible endpoint — the single source of truth (§11.4.6, mirrors CONST-036 "no hardcoded model lists"):

```
helixagent_models() {   # echoes model ids, one per line
  local base="$1" keyvar="$2" key="${!keyvar:-}"
  curl -s --max-time 8 \
    --config <(printf 'header = "Authorization: Bearer %s"\n'
        "$key") \
    "${base%/}/models" \
  | jq -r '.data[].id'          # OpenAI /v1/models shape: {data:
        [{id,...}]}
}
```

- **strong** model = `helix-debate` if present (the AI Debate Ensemble — best quality), else the first id; **fast** = `helix-llm` if present, else the cheapest/first. Selection is data-driven from the returned ids, never a static list. In `--multi` mode, feed ALL returned ids into `model_verify.py` / `providers_generate.py` so one alias per model is created (`helixagent`, `helixagent2`, …) exactly like other multi-providers.
- If `/v1/models` is unreachable (server down) → emit the record with `status: resolved` but let verification mark it `unverified`, and default the model pair to the `overrides.json` pins so the alias still exists (an honest "installed but not yet verified" state, not a bluff PASS).

- The auth token comes from the **key-var NAME** `HELIXAGENT_API_KEY` resolved at run time (same non-secret discipline as every other alias) — the operator puts `export HELIXAGENT_API_KEY=…` in `~/api_keys.sh`, OR the detector calls `helixagent --generate-api-key --api-key-env-file ~/api_keys.sh` to bootstrap it (documented, opt-in — never auto-writes secrets silently).

4.c base-URL + auth record for a local HelixAgent

The written `helixagent.env` (non-secret) will contain:

```
CMA_PROVIDER_ID=helixagent
CMA_PROVIDER_KEYVAR=HELIXAGENT_API_KEY        # NAME only, resolved at lau
CMA_PROVIDER_TRANSPORT=router                 # OpenAI-compat → route via
CMA_PROVIDER_BASE_URL=http://localhost:8100/v1
CMA_PROVIDER_MODEL=helix-debate               # strong (AI Debate Ensemble
CMA_PROVIDER_FAST_MODEL=helix-llm             # fast (HelixLLM)
CMA_PROVIDER_CONFIG_DIR=/home/<u>/.claude-prov-helixagent
CMA_PROVIDER_CONTEXT_LIMIT=128000
CMA_PROVIDER_MAX_OUTPUT=8192
CMA_PROVIDER_ALIAS=helixagent
```

Host/port/binary/key-var are all env-overridable (`CMA_HELIXAGENT_*`) so the detector is reusable/decoupled (§11.4.28) — no project-specific paths baked in. For the native HelixLLM `https://localhost:8443` endpoint (self-signed TLS), the detector can additionally trust the cert exactly as helix_agent's `configureHelixLLMTLS` does (point `SSL_CERT_FILE` at the combined CA bundle) — optional, only if the operator selects the native endpoint.

4.d Comprehensive verification test (mirrors existing alias tests)

Add **`scripts/tests/verify_helixagent_live.sh`**, modelled 1:1 on `verify_providers_live.sh` (§1.6), read-only + honest-SKIP when HelixAgent is not on PATH / not running (§11.4.3 — never a faked PASS):

1. **PATH + record checks** — `command -v helixagent` present; a `helixagent.env` exists with all required `CMA_PROVIDER_*` fields and **no secret values** (reuse the structural "only `CMA_PROVIDER_*=` lines" assert).
2. **Live `/v1/models`** — real `curl` to `${BASE_URL}/models` returns HTTP 200 and a non-empty `.data[].id` list that **includes the alias's `CMA_PROVIDER_MODEL`**; capture the raw JSON to `proof/80-helixagent-models.txt` (redacted). (§11.4.69 sink-side positive evidence — a non-empty enumerated model list, not a config-only pass.)
3. **Real chat round-trip (no mock, §11.4.50/CONST-035)** — POST to `${BASE_URL}/chat/completions` with model `helix-debate` and a

deterministic prompt (`"Reply with exactly the word HELIXOK"`), assert the response body contains a real assistant completion (`choices[0].message.content` non-empty and matches `HELIXOK`), capture to `proof/81-helixagent-chat.txt`. This is the anti-bluff proof the ensemble genuinely answers — not just that a port is open.
4. **Alias launch coherence** — `cma_run_provider helixagent -p "…" --output-format json` through the router path returns a real model response (reuse the `verify_providers_live.sh` launch harness + secret-redactor + throwaway cwd).
5. **Negative case** — a neutral prompt must NOT falsely match the engagement-marker regex (copy the §1.6 negative-case block) — proves the analyzer itself can't bluff (§11.4.107(10)).
6. Optionally wire strategy-1 **LLMsVerifier** at the HelixAgent endpoint (`providers-verify.sh --provider helixagent --model helix-debate --base-url …/v1`) so the same "Do you see my code?" contract runs.

All evidence lands under `scripts/tests/proof/8x-helixagent-*.txt`; the test is NOT named `test_*.sh` (so `run-all.sh` won't auto-pick a live network test), and SKIPs cleanly when preconditions are absent — matching the repo convention.

Also extend the hermetic **`test_providers.sh`** with a fake-`helixagent` on the `fakebin` PATH + a fake `/v1/models` responder, asserting the detector produces the expected record and env/alias — so detection has both a hermetic gate AND a live proof (four-layer, §11.4.4(b)).

---

5. Web research needed

The design is derived entirely from the two codebases (no external dependency for correctness), so no web research was required for THIS report. For the IMPLEMENTATION, two best-practice cross-checks are worth a quick §11.4.99 latest-source verification before coding:

1. **OpenAI `/v1/models` + `/v1/chat/completions` response shapes** — confirm `{"data":[{"id":…}]}` and `choices[0].message.content` against the current OpenAI API reference (HelixAgent claims OpenAI-compatibility; verify the exact JSON before asserting on it).
2. **CLI provider auto-detection patterns** — how tools like `ollama`, `llama.cpp`/`llama-server`, and `aider`/`opencode` detect a local OpenAI-compatible server (PATH probe + a `/models` or `/health` liveness ping, with graceful "installed-but-not-running" state) — to confirm the PATH-gate-then-liveness-ping shape in §4.a matches established practice.

(No web fetches were performed in this session; the two items above are flagged as pre-implementation verification, not completed research.)

---

6. Top 3 risks

1. **Transport mismatch (OpenAI-compat vs Anthropic-native).** HelixAgent's `/v1` is OpenAI-shaped; Claude Code speaks Anthropic. The alias MUST route via `ccr` (`transport: router`) — if mistakenly pinned `native` it will send Anthropic-format requests to an OpenAI endpoint and fail at launch. Mitigate: default `overrides.json` pins `router`; the live test's chat round-trip (§4.d step 3) catches a wrong transport immediately.
2. **PATH-present ≠ server-running.** `command -v helixagent` succeeding does NOT mean the `:8100/v1` server is up, nor that `HELIXAGENT_API_KEY` is set. Registering a "verified" alias off a mere PATH hit would be a §11.4 PASS-bluff. Mitigate: PATH gate registers the alias, but model-enumeration + verdict come from a real `/v1/models` + chat probe; server-down → honest `unverified`, not a fake pass.
3. **LLMsVerifier re-incorporation drift.** The vendored copy is 1 commit behind `origin/main`, and strategy-1 expects a `bin/model-verification` binary that is not committed — a bump without rebuilding the consumed binaries leaves the toolkit exercising STALE verifier code (§11.4.108 SOURCE→ARTIFACT gap). Mitigate: the §2.3 procedure force-rebuilds `.local-cache/code-verification`
    - `bin/model-verification` and re-runs `test_verify_providers.sh` + `verify_providers_live.sh` before committing the gitlink bump.

Report path: `/home/milos/Factory/projects/tools_and_research/helix_code/docs/research/07.2026/11_claude_toolkit/11_claude_toolkit.md`